

A PROJECT REPORT ON

FLOOD EARLY WARNING AND EVACUATION SUPPORT SYSTEM

Domain: Disaster Management / GIS

Submitted in partial fulfilment of the requirements for the award of the degree of

B.Sc. / B.E. / MCA – COMPUTER SCIENCE

Submitted by

STUDENT NAME

(Register No: XXXXXXXXX)

Under the guidance of

GUIDE NAME, M.Sc., M.Phil.,

Assistant Professor

DEPARTMENT OF COMPUTER SCIENCE

COLLEGE NAME

2025 – 2026

CONTENTS

CHAPTER	TITLE	PAGE
I	INTRODUCTION	1
	1.1 History of Domain	
	1.2 Overview of the Project	
	1.3 Motivation	
	1.4 Aim of the Project	
	1.5 Objectives	
	1.6 Literature Review	
II	SOFTWARE PROJECT PLAN	5
	2.1 Existing System	
	2.2 Proposed System	
	2.3 Project Schedule	
III	CUSTOMER REQUIREMENT DETERMINATION	7
	3.1 Functional Requirements	
	3.2 Non-functional Requirements	
	3.3 User Requirements	
	3.4 System Requirements	
IV	SOFTWARE REQUIREMENT SPECIFICATION	9
	4.1 Hardware Specification	
	4.2 Software Specification	
V	SYSTEM ANALYSIS	10
	5.1 Feasibility Study	

	5.2 About the Software	
VI	SYSTEM DESIGN	14
	6.1 UML Diagram	
	6.2 System Architecture	
	6.3 Data Flow Diagram	
	6.4 ER Diagram	
	6.5 Table Design	
	6.6 Algorithm Design	
	6.7 Modules	
VII	IMPLEMENTATION AND TESTING	23
	7.1 Introduction	
	7.2 Implementation	
	7.3 Test Cases	
	7.4 Source Code	
	7.5 Screen Layout	
VIII	CONCLUSION AND FUTURE ENHANCEMENT	30
	8.1 Conclusion	
	8.2 Future Enhancement	
IX	BIBLIOGRAPHY	31

CHAPTER I

INTRODUCTION

Floods are among the most frequent and destructive natural disasters in India. Delta and river-basin districts such as those along the Cauvery, Vaigai and Thamirabarani rivers and coastal cities like Chennai face flooding during the north-east and south-west monsoons. Heavy rainfall over a short period, release of surplus water from dams, and blocked drainage can raise river and canal levels within hours.

Although rainfall forecasts and river gauge readings are collected by different departments, the information is scattered and is not converted into easy, location-specific warnings for residents. Warnings often reach people too late, are issued in English, or do not tell people where to go. Many residents do not know the location of the nearest relief shelter or which roads are safe.

This project, FloodWatch, combines rainfall, river level and dam release data to predict flood risk by area using machine learning, displays risk zones on a GIS map, sends multilingual alerts by SMS and Telegram to residents of affected zones, and shows the nearest safe shelter and an evacuation route that avoids flooded roads.

LIMITATIONS

1. Warnings often reach residents too late.
2. Data from different departments is scattered.
3. People do not know the nearest safe shelter.
4. Alerts are not specific to the resident's area or language.

1.1 History of Domain

Flood forecasting historically relied on manual river gauge observations and rating curves relating water level to discharge. The Central Water Commission (CWC) of India began flood forecasting in 1958 on the Yamuna and now operates forecasting stations on major rivers. Hydrological models such as HEC-HMS and MIKE 11 simulate rainfall-runoff and river flow.

Geographic Information Systems (GIS) since the 1990s have made it possible to map flood-prone areas using elevation and land use data. In recent years, machine learning models such as Random Forests and LSTM networks have been shown to forecast river levels accurately from historical data, and Google's Flood Hub (2022) provides AI-based riverine flood forecasts for India and other countries.

1.2 Overview of the Project

FloodWatch ingests rainfall data from weather APIs and rain gauges, river level readings from gauging stations or IoT sensors, dam release announcements and weather forecasts. The time series are normalised and stored in PostgreSQL with PostGIS. A Random Forest model and an LSTM model predict the river level at each gauge for the next 6, 12 and 24 hours.

Predicted levels are compared with warning and danger levels and combined with ward-level elevation and drainage information to classify each ward into low, medium or high risk. The risk zones are overlaid on a Leaflet map. Residents of medium and high risk zones receive SMS and Telegram alerts in Tamil and English with the nearest shelter, computed using PostGIS distance queries and a safe route that avoids flooded segments.

1.3 Motivation

During past floods, many people were trapped because they did not receive warnings in time or did not know where to go.

- **Motivation:** To give every resident a timely, area-specific warning in their language along with a clear safe destination.
- **Outcome:** Faster evacuation, reduced loss of life and property, and better coordination for district officials.

1.4 Aim of the Project

The project is entitled as “**FLOOD EARLY WARNING AND EVACUATION SUPPORT SYSTEM**”. The aim of this project is to develop an integrated flood early

warning system that predicts flood risk by area from hydro-meteorological data, visualises it on a map, and supports evacuation through targeted multilingual alerts and shelter routing.

1.5 Objectives

- To ingest and store rainfall, river level and dam release time series.
- To predict river levels using Random Forest and LSTM models.
- To classify wards into risk zones and display them on a GIS map.
- To find the nearest shelter and a safe route for each zone.
- To send zone-wise SMS/Telegram alerts in Tamil and English.

1.6 Literature Review

Mosavi, Ozturk and Chau (2018): Reviewed machine learning models for flood prediction and found that ANN, SVM, decision trees and ensemble methods provide accurate short-term flood forecasts and are increasingly preferred over physical models for speed.

Kratzert et al. (2018): Showed that LSTM networks can model rainfall-runoff relationships across hundreds of catchments with accuracy comparable to or better than traditional hydrological models.

Breiman (2001): Introduced Random Forests, an ensemble of decision trees trained on bootstrap samples with random feature selection, robust to overfitting and suitable for nonlinear regression.

Nevo et al. (2022): Described Google's operational flood forecasting system that combines LSTM-based stage forecasting with inundation models and delivers alerts to millions of people in India and Bangladesh.

Central Water Commission: The CWC publishes warning and danger levels for river gauge stations, which are used in this project as reference thresholds for risk classification.

NDMA Guidelines on Management of Floods (2008): The National Disaster Management Authority guidelines emphasise early warning dissemination to the last mile and identification of safe shelters.

CHAPTER II

SOFTWARE PROJECT PLAN

2.1 Existing System

In the existing system, rainfall and river data are collected by meteorological and water resource departments and shared with district administration through reports and phone calls. Warnings are announced through television, newspapers and loudspeakers. Shelter information is announced separately and is not linked to the resident's location.

DISADVANTAGES

- Delay in converting data into public warnings.
- Warnings are general and not area-specific.
- No prediction of future river levels for small rivers and canals.
- Residents do not know the nearest shelter or safe road.
- No feedback on evacuation outcomes.

2.2 Proposed System

The proposed FloodWatch system integrates data sources, predicts river levels with machine learning, classifies wards into risk zones, and shows them on a live map. It automatically sends zone-specific alerts in Tamil and English with the nearest shelter and a safe evacuation route, and records outcomes to improve the model.

ADVANTAGES

- Predictions 6 to 24 hours in advance.
- Area-specific, multilingual alerts.
- Nearest shelter and safe route for each resident.
- Single dashboard for officials.
- Model improves with recorded outcomes.

2.3 Project Schedule

The development of FloodWatch follows the iterative waterfall model. Each phase produces a deliverable that is reviewed before the next phase begins, and feedback from testing is fed back into design and implementation. The planned schedule is shown below.

S.No	Phase	Duration
1	Requirement analysis and problem study	Week 1 - 2
2	Literature survey and existing system study	Week 3 - 4
3	Dataset collection / hardware procurement	Week 5 - 6
4	System design (DFD, UML, ER, table design)	Week 7 - 8
5	Implementation of modules	Week 9 - 12
6	Testing and model evaluation	Week 13 - 14
7	Documentation and final review	Week 15 - 16

CHAPTER III

CUSTOMER REQUIREMENT DETERMINATION

3.1 Functional Requirements

Functional requirements define the specific features and operations a system must perform to meet business and user needs. They describe what the system should do and how it interacts with users or other systems. They focus on the behaviour and functionality of the system and represent features that can be directly observed and tested in the final product. The functional requirements of the FloodWatch system are listed below.

- **Data ingestion:** Fetch rainfall, river level, dam release and forecast data.
- **Storage:** Store time series and geographic layers.
- **Prediction:** Predict river levels for 6, 12, 24 hours.
- **Risk classification:** Classify wards as low/medium/high risk.
- **Map display:** Overlay risk zones, gauges and shelters on a map.
- **Shelter routing:** Find the nearest open shelter and safe route.
- **Alerts:** Send SMS/Telegram alerts by zone.
- **Outcome recording:** Record actual flooding to retrain the model.

3.2 Non-functional Requirements

Non-functional requirements determine the performance standards and quality attributes of software, e.g. system usability, effectiveness, security, scalability, etc. While functional requirements determine what the system does, non-functional requirements describe HOW the system does it. If an application does not meet non-functional requirements, it continues to perform its basic functions; however, it will not be able to provide a good user experience. Therefore, non-functional requirements are as important as functional requirements for successful product adoption.

- **Timeliness:** Predictions refreshed every hour; alerts sent within 5 minutes.
- **Reliability:** Fallback to threshold rules if the model service fails.
- **Scalability:** Send alerts to 1 lakh subscribers per hour.

- **Usability:** Map readable on mobile phones.
- **Language:** Alerts in Tamil and English.
- **Accuracy:** River level RMSE below 0.25 m for 12-hour forecasts.

3.3 User Requirements

- Residents get early alerts about their own area
- Residents know where to go and which route is safe
- Officials see risk zones and shelter capacity
- Officials can manually issue alerts
- Historical flood records are available

3.4 System Requirements

- Python data pipeline and ML models
- PostgreSQL with PostGIS
- Leaflet web map
- Twilio SMS and Telegram Bot API
- Weather and river data APIs or sensors

CHAPTER IV

SOFTWARE REQUIREMENT SPECIFICATION

4.1 Hardware Specification

System	: Intel i5 Processor 2.4 GHz or above
Hard Disk	: 512 GB
RAM	: 16 GB
Monitor	: 15" colour monitor
Field sensors (optional)	: Ultrasonic water level sensor with ESP32 and GSM

4.2 Software Specification

Operating System	: Ubuntu 22.04 / Windows 10
Coding Language	: Python 3.10, JavaScript
ML Libraries	: scikit-learn, TensorFlow (LSTM)
GIS	: GeoPandas, PostGIS, Leaflet.js
Back End	: Flask
Database	: PostgreSQL + PostGIS
Alerts	: Twilio SMS, Telegram Bot API

CHAPTER V

SYSTEM ANALYSIS

5.1 Feasibility Study

A feasibility study is carried out to find whether the proposed FloodWatch system is worth developing. Three key considerations are involved in the feasibility analysis: technical, economic and operational feasibility.

TECHNICAL FEASIBILITY

Historical rainfall data (IMD), river level data (India-WRIS) and open map data are available. Python ML and GIS libraries are mature. The project is technically feasible.

ECONOMIC FEASIBILITY

Software is open source; SMS costs are small compared with the cost of flood damage. District servers can host the system. It is economically feasible.

OPERATIONAL FEASIBILITY

Residents only need to register their mobile number and ward. Officials use a simple map dashboard. The system fits existing disaster management procedures, so it is operationally feasible.

5.2 About the Software

PYTHON

Python is a high-level, interpreted, general-purpose programming language created by Guido van Rossum and first released in 1991. Its design philosophy emphasises code readability through significant indentation, and its simple syntax allows developers to express concepts in fewer lines of code than languages such as C++ or Java.

Python supports multiple programming paradigms including procedural, object-oriented and functional programming. It has a very large standard library and an ecosystem of third-party packages available through the Python Package Index (PyPI),

which makes it the most popular language for data science, machine learning, automation and back-end web development.

- Easy to learn, read and maintain
- Huge library support for AI, ML and data processing (NumPy, Pandas, scikit-learn)
- Platform independent - runs on Windows, Linux and macOS
- Strong community support and documentation

SCIKIT-LEARN

Scikit-learn is a free machine learning library for Python built on NumPy, SciPy and Matplotlib. It provides simple and efficient tools for predictive data analysis including classification, regression, clustering and dimensionality reduction.

It contains implementations of popular algorithms such as Random Forest, Support Vector Machine, Logistic Regression, k-Nearest Neighbours and Isolation Forest, along with utilities for train-test splitting, cross-validation, feature scaling, pipelines and evaluation metrics such as accuracy, precision, recall and F1-score.

- Consistent fit / predict API for all models
- Built-in preprocessing and model selection tools
- Well documented and production tested

TENSORFLOW AND KERAS

TensorFlow is an open-source end-to-end machine learning platform developed by the Google Brain team. It provides a flexible ecosystem of tools, libraries and community resources that lets researchers build and deploy machine learning powered applications on servers, browsers and mobile devices.

Keras is the high-level API of TensorFlow. It offers simple and consistent interfaces for building neural networks layer by layer, which reduces the number of user actions required for common use cases. Pre-trained models such as MobileNetV2, ResNet

and EfficientNet are available in `keras.applications` and can be fine-tuned using transfer learning on a small custom dataset.

- Supports CPU, GPU and TPU execution
- Transfer learning with pre-trained ImageNet models
- TensorFlow Lite for running models on mobile phones
- TensorBoard for visualising training progress

GEOPANDAS

GeoPandas extends the Pandas data analysis library to support geographic data. It combines Shapely for geometric operations and Fiona for file access, allowing shapefiles and GeoJSON to be read, joined spatially and plotted.

POSTGRESQL

PostgreSQL is a powerful, open-source object-relational database system with over 35 years of active development. It has earned a strong reputation for reliability, feature robustness and performance.

PostgreSQL supports ACID transactions, foreign keys, views, triggers, stored procedures and JSON data types. Its extension system allows additional capabilities such as PostGIS for geographic data and pgvector for similarity search.

- Fully ACID compliant
- Advanced SQL and JSON support
- Extensible with PostGIS and other extensions

POSTGIS

PostGIS is a spatial database extender for PostgreSQL. It adds support for geographic objects, allowing location queries to be run in SQL, such as finding all shelters within 5 km of a point or all villages that fall inside a flood polygon.

PostGIS provides spatial indexes (GiST), geometry and geography data types and hundreds of spatial functions like `ST_Distance`, `ST_Within` and `ST_Buffer`.

LEAFLET.JS

Leaflet is the leading open-source JavaScript library for mobile-friendly interactive maps. It is lightweight (about 42 KB) and has all the mapping features most developers need, such as tile layers, markers, pop-ups, polylines and GeoJSON layers.

Leaflet works with free OpenStreetMap tiles, so maps can be shown without paying for a commercial map API.

FLASK

Flask is a lightweight WSGI web application framework written in Python. It is classified as a micro-framework because it does not require particular tools or libraries; it has no database abstraction layer or form validation built in, but extensions can add these features as needed.

Flask provides URL routing, request handling, Jinja2 templating and a development server. Because it is small and simple, it is ideal for exposing a trained machine learning model as a REST API or building a small web application quickly.

- Minimal and easy to learn
- Jinja2 templates for HTML pages
- Large set of extensions (Flask-Login, Flask-SQLAlchemy, Flask-CORS)

TWILIO SMS / TELEGRAM BOT API

Twilio is a cloud communications platform that offers APIs for sending SMS, making voice calls and sending WhatsApp messages from code. The Telegram Bot API is a free HTTP-based interface for sending messages to Telegram users and groups.

These services let the system deliver alerts directly to users' phones within seconds, even if they are not using the application at that time.

CHAPTER VI

SYSTEM DESIGN

System design is the process of defining the architecture, components, modules, interfaces and data of a system to satisfy the specified requirements. The design of FloodWatch is presented using UML use case diagram, system architecture, data flow diagrams, ER diagram and table design.

6.1 UML Diagram

USE CASE DIAGRAM

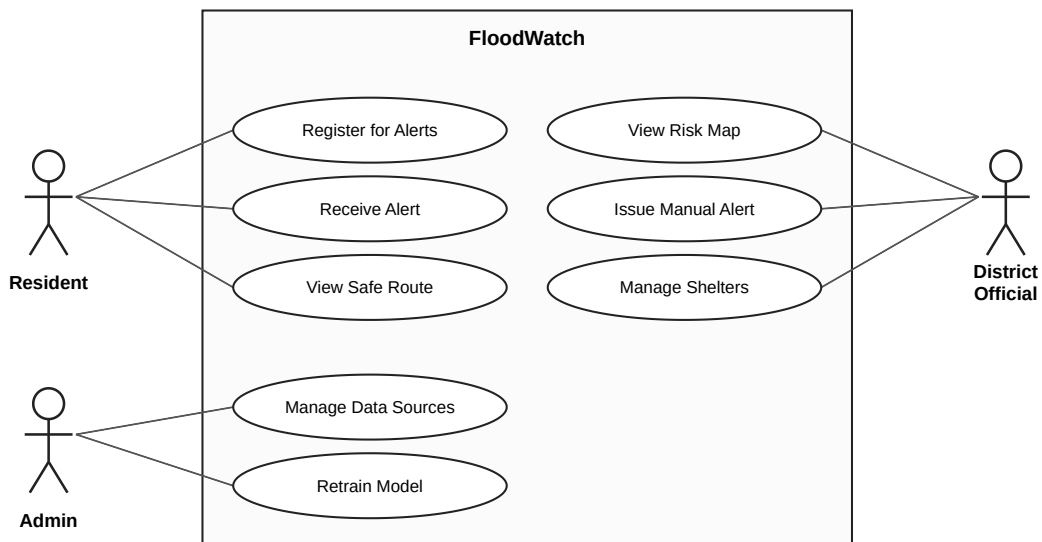


Fig 1: Use case diagram of FloodWatch

6.2 System Architecture

FloodWatch is organised as a data pipeline feeding a prediction service, with a GIS-enabled database and map-based front end. Alerts are dispatched by a separate worker so that large volumes do not slow down the dashboard.

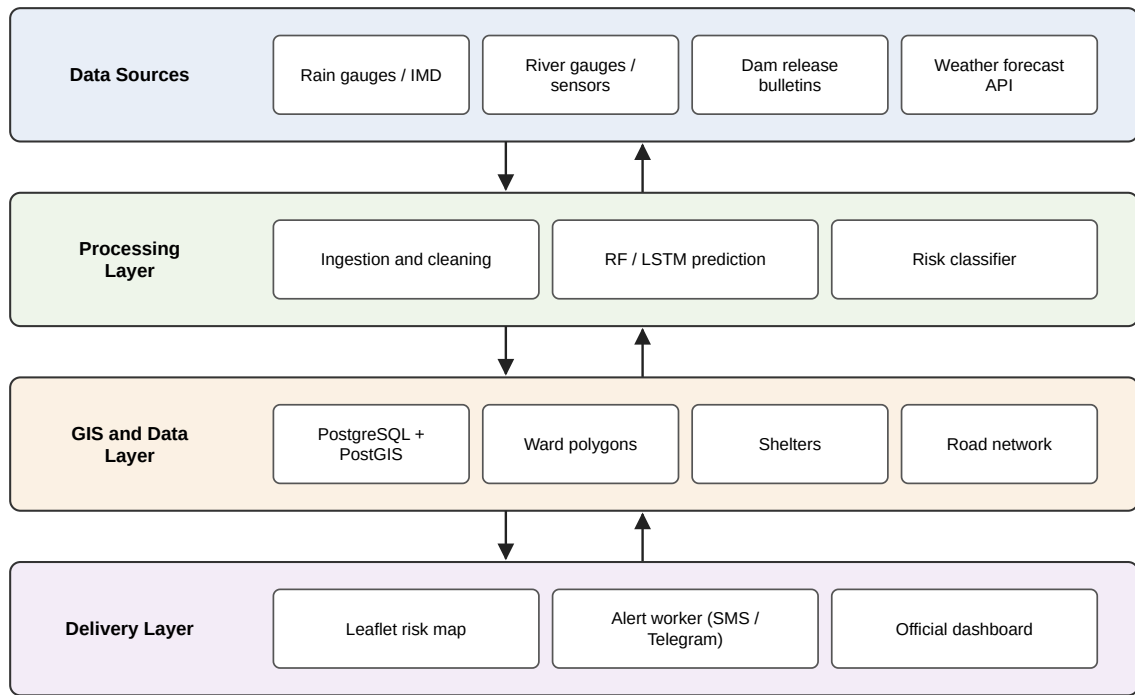


Fig 2: System architecture

6.3 Data Flow Diagram

A Data Flow Diagram (DFD) is a graphical representation of the flow of data through an information system. The Level 0 DFD (context diagram) shows the whole system as a single process with its external entities, and the Level 1 DFD breaks it into its major processes and data stores.

LEVEL 0 DFD (CONTEXT DIAGRAM)

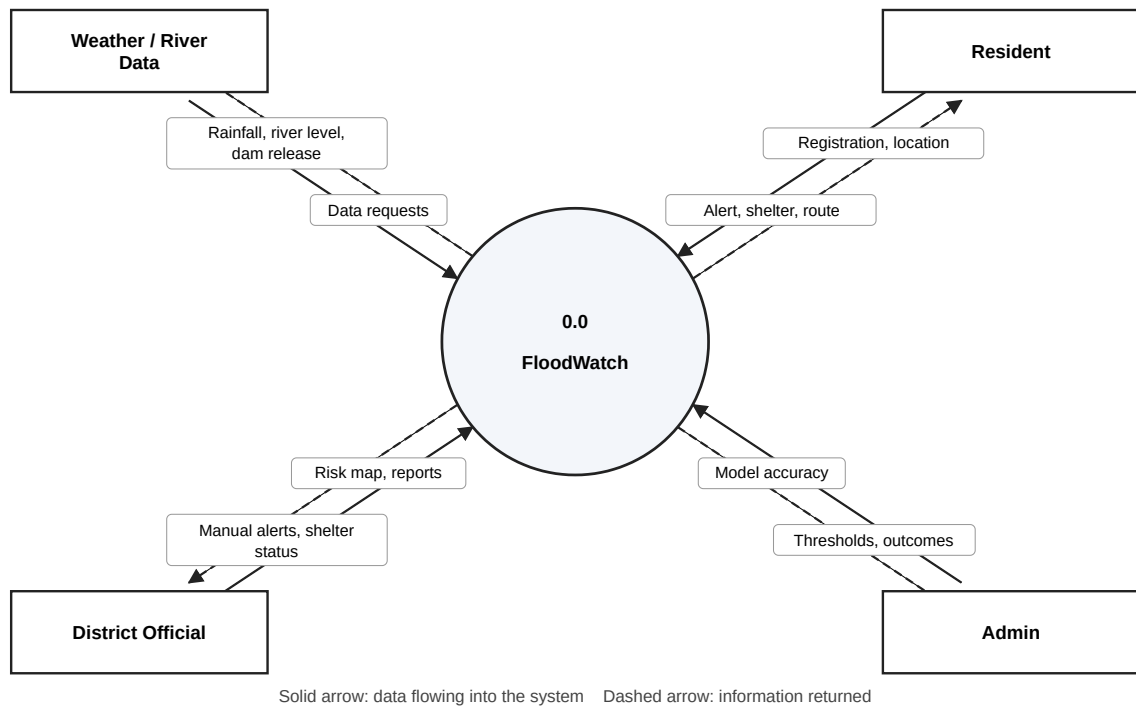


Fig 3: Level 0 data flow diagram

LEVEL 1 DFD

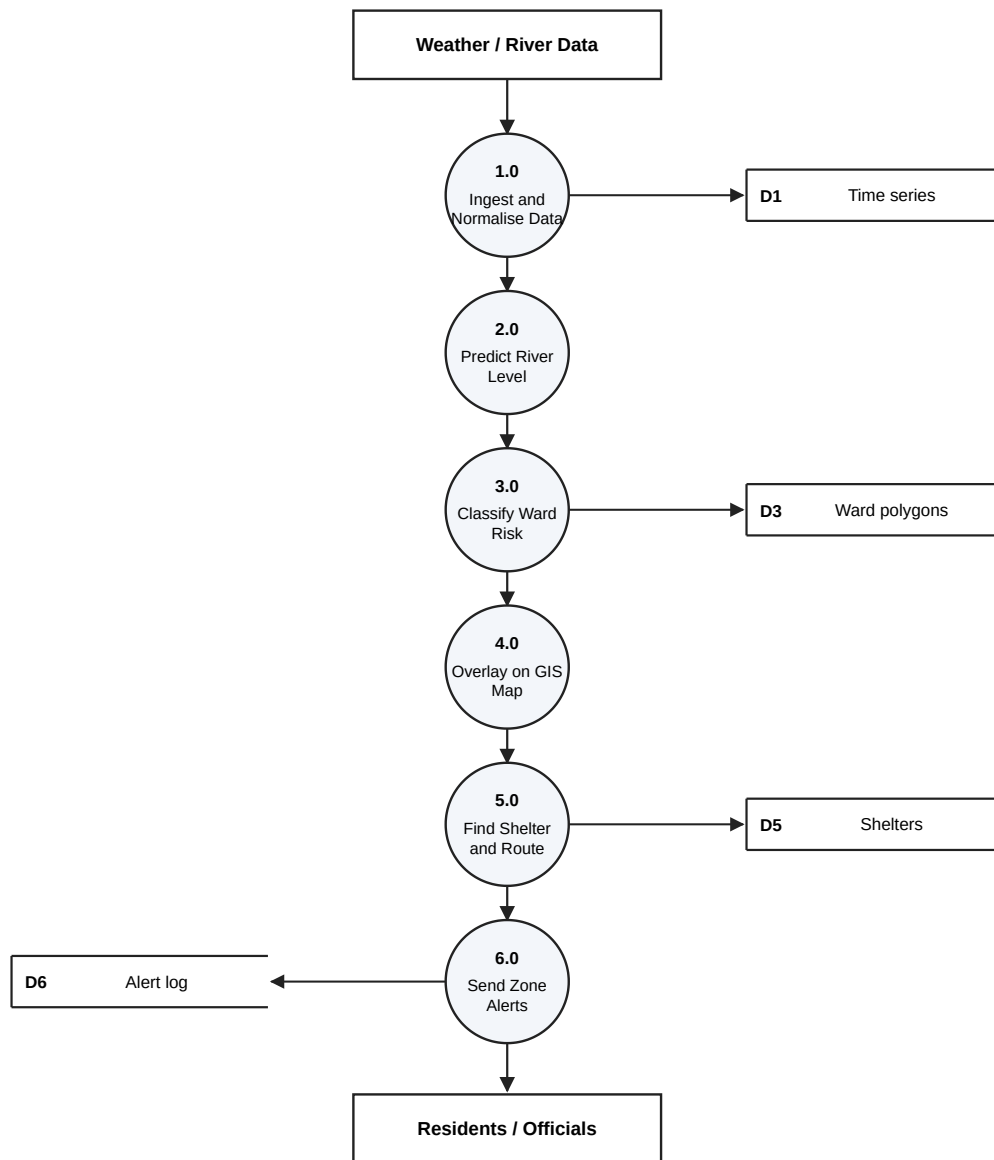


Fig 4: Level 1 data flow diagram

6.4 ER Diagram

The Entity Relationship (ER) diagram shows the entities stored in the database, their key attributes and the relationships between them. 1:N indicates a one-to-many relationship.

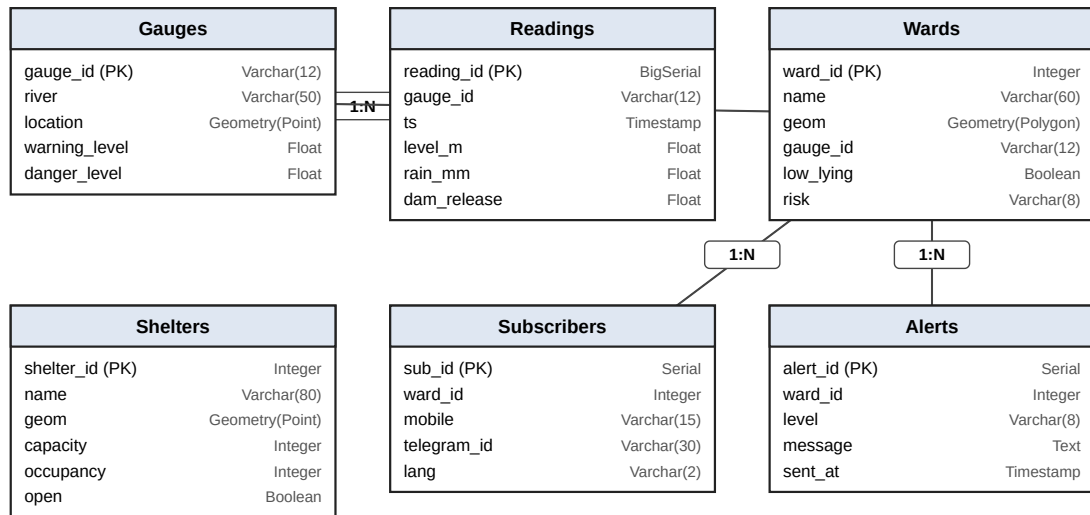


Fig 5: Entity relationship diagram

6.5 Table Design

GAUGES

FIELD NAME	DATA TYPE	DESCRIPTION
gauge_id	Varchar(12)	Primary key
river	Varchar(50)	River / canal
location	Geometry(Point)	Gauge location
warning_level	Float	Warning level (m)
danger_level	Float	Danger level (m)

READINGS

FIELD NAME	DATA TYPE	DESCRIPTION
reading_id	BigSerial	Primary key
gauge_id	Varchar(12)	Foreign key to Gauges
ts	Timestamp	Time
level_m	Float	River level
rain_mm	Float	Rainfall last hour
dam_release	Float	Upstream release (cumecs)

WARDS

FIELD NAME	DATA TYPE	DESCRIPTION
ward_id	Integer	Primary key
name	Varchar(60)	Ward / village
geom	Geometry(Polygon)	Boundary
gauge_id	Varchar(12)	Nearest gauge
low_lying	Boolean	Low lying area
risk	Varchar(8)	low / medium / high

SHELTERS

FIELD NAME	DATA TYPE	DESCRIPTION
shelter_id	Integer	Primary key
name	Varchar(80)	School / hall name
geom	Geometry(Point)	Location
capacity	Integer	Capacity
occupancy	Integer	Current occupancy
open	Boolean	Open status

SUBSCRIBERS

FIELD NAME	DATA TYPE	DESCRIPTION
sub_id	Serial	Primary key
ward_id	Integer	Foreign key to Wards
mobile	Varchar(15)	Mobile
telegram_id	Varchar(30)	Telegram chat id
lang	Varchar(2)	ta / en

ALERTS

FIELD NAME	DATA TYPE	DESCRIPTION
alert_id	Serial	Primary key
ward_id	Integer	Foreign key to Wards

level	Varchar(8)	medium / high
message	Text	Alert text
sent_at	Timestamp	Time

6.6 Algorithm Design

The early warning algorithm runs every hour and combines prediction, spatial analysis and alert dispatch.

1. **Ingest rainfall, river gauge and weather forecast data** – Data from APIs and sensors is fetched hourly.
2. **Normalize and store time series** – Readings are resampled to hourly values; missing values are interpolated.
3. **Predict river level using LSTM/Random Forest** – Features: last 24 hours of level, cumulative rainfall (6/24/72 h), forecast rainfall, upstream dam release.
4. **Classify risk zones (low/medium/high)** – High if predicted level $\geq$ danger level or ($\geq$ warning level and ward is low-lying); medium if $\geq$ warning level; else low.
5. **Overlay risk on GIS map** – Ward polygons are coloured by risk on the Leaflet map.
6. **Find nearest shelter and safe route** – PostGIS ST_Distance finds the nearest open shelter with capacity; Dijkstra on the road graph excludes segments inside high-risk polygons.
7. **Send SMS/Telegram alerts by zone** – Messages in the subscriber's language with shelter and route link.
8. **Record outcomes to improve the model** – Actual flooding is recorded and used for retraining.

PSEUDO CODE

```

EVERY HOUR:
  INGEST(rain, river, dam, forecast)
  FOR each gauge g:
    X <- FEATURES(g, last 24 h, rain sums, forecast, dam release)
    lvl[g] <- {6h, 12h, 24h: MODEL[h].predict(X)}

```

```

FOR each ward w:
  p <- MAX(lvl[w.gauge])
  w.risk <- HIGH if p >= danger or (p >= warning and w.low_lying)
              MEDIUM if p >= warning ELSE LOW
  IF w.risk changed to MEDIUM/HIGH:
    s <- NEAREST_OPEN_SHELTER(w.centroid)
    route <- DIJKSTRA(road_graph minus HIGH-risk polygons, w.centroid, s)
    SEND_ALERTS(w.subscribers, w.risk, s, route)

```

6.7 Modules

The modules are grouped according to the Resident, District Official and Admin users.

MODULES

RESIDENT

1. REGISTER FOR ALERTS
2. RECEIVE ALERT
3. VIEW SAFE ROUTE

DISTRICT OFFICIAL

1. VIEW RISK MAP
2. ISSUE MANUAL ALERT
3. MANAGE SHELTERS

ADMIN

1. MANAGE DATA SOURCES
2. RETRAIN MODEL

RESIDENT

REGISTER FOR ALERTS

Residents register their mobile number or Telegram, preferred language and home location (ward selected on map).

RECEIVE ALERT

When their ward becomes medium or high risk, residents receive an alert with expected time, nearest shelter and helpline numbers.

VIEW SAFE ROUTE

A link in the alert opens the map with the route from their location to the nearest open shelter avoiding flooded roads.

DISTRICT OFFICIAL

VIEW RISK MAP

Officials see current and forecast risk zones, gauge levels and shelter occupancy on a map.

ISSUE MANUAL ALERT

Officials can issue or cancel alerts for selected wards.

MANAGE SHELTERS

Officials update shelter status (open/closed) and occupancy.

ADMIN

MANAGE DATA SOURCES

Admin configures APIs, sensor feeds and threshold levels.

RETRAIN MODEL

Admin records actual flood outcomes and retrains the prediction model.

CHAPTER VII

IMPLEMENTATION AND TESTING

7.1 Introduction

FloodWatch was implemented with Python scripts scheduled using APScheduler for ingestion and prediction, a Flask API, PostgreSQL with the PostGIS extension, and a Leaflet front end. Alerts are sent through Twilio for SMS and the Telegram Bot API, with message templates in Tamil and English.

7.2 Implementation

The models were trained on hourly river level and rainfall data for a sample river basin covering five monsoon seasons, with data for 2016-2020 used for training and 2021 used for testing. The 2015 Chennai floods and 2021 November rainfall events were used as case studies for validating alert timing.

Testing is a process of running software with the intention of finding errors. The software which has been developed has to be tested to prove its validity. Testing is considered to be the least creative phase of the whole cycle of system design; in the real sense it is the phase which helps to bring out the creativity of the other phases. Testing a program is the first step in the debugging process.

A good testing suite is vital because all possible execution paths inside the code must be tested in order to have any hope that it is bug free. Testing of the system is done to ensure the integrity of the system. The aim of testing is to prove that the developed system addresses the predefined processing requirements and will perform reliably and efficiently during operation.

PURPOSES OF TESTING

- To affirm the quality of the project.
- To find and eliminate any errors from previous stages.
- To validate the software against its requirements.

- To ensure the reliability of the system.

UNIT TESTING

In the lines of this strategy all the individual functions and modules were put to the test independently. By following this strategy all the errors in coding were identified and corrected. This method was applied in combination with white box and black box testing techniques to find the errors in each module.

INTEGRATION TESTING

This testing strategy has two different approaches, namely the top-down approach, in which integration is carried out from the top-level module to the bottom, and the bottom-up approach, in which integration is carried out from the low-level modules to the top. The modules were tested using the bottom-up approach by introducing stubs for the top-level functions. This test was used to identify errors in the interfaces and in passing parameters between functions.

VALIDATION TESTING

The main aim of this testing is to verify that the software system does what it was designed for. Alpha testing and beta testing were conducted to ensure the validity of the system. Alpha testing was done at the developer's site and beta testing was carried out by sample end users.

SYSTEM TESTING

The idea of applying this strategy was to put the system through a series of different tests whose primary purpose is to fully exercise the computer-based system. Although each test has a different purpose, all work to verify that all system elements have been properly integrated and perform their allocated functions. Security testing and performance testing were also conducted.

ACCEPTANCE TESTING

Acceptance testing involves planning and execution of functional tests, performance tests and stress tests to verify that the implemented system satisfies its

requirements. Typically, the acceptance test incorporates test cases developed during unit testing and integration testing.

7.3 Test Cases

Test ID	Test Description	Input	Expected Output	Result
TC01	Data ingestion	Hourly API data	Readings stored in database	Pass
TC02	12-hour prediction	Test period data	RMSE < 0.25 m	Pass
TC03	High risk classification	Predicted level above danger level	Ward coloured red	Pass
TC04	Nearest shelter	Ward centroid	Nearest open shelter returned	Pass
TC05	Safe route	Road passes through flooded zone	Route avoids flooded road	Pass
TC06	Tamil alert	Subscriber lang = ta	SMS received in Tamil	Pass
TC07	Shelter full	Occupancy = capacity	Next nearest shelter suggested	Pass
TC08	Model failure fallback	Model service stopped	Threshold rules used, alert still sent	Pass

PERFORMANCE EVALUATION

River level prediction error on the 2021 test season:

Model	6 h RMSE (m)	12 h RMSE (m)	24 h RMSE (m)
Persistence (last value)	0.21	0.38	0.61
Linear regression	0.17	0.29	0.47
Random Forest	0.12	0.21	0.36
LSTM (proposed)	0.10	0.18	0.31

7.4 Source Code

predict_levels.py

```
import pandas as pd, numpy as np, joblib
from sqlalchemy import create_engine

eng = create_engine("postgresql://flood:secret@localhost/floodwatch")
MODELS = {h: joblib.load(f"models/rf_{h}.pkl") for h in (6, 12, 24)}

def features(gauge_id):
    df = pd.read_sql("""SELECT ts, level_m, rain_mm, dam_release FROM readings
                        WHERE gauge_id=%(g)s AND ts > now() - interval '3 days'
ORDER BY ts""",
                    eng, params={"g":
gauge_id}).set_index("ts").resample("1h").mean().interpolate()
    last = df.iloc[-1]
    f = {f"lvl_lag{i}": df.level_m.iloc[-1 - i] for i in range(0, 24, 3)}
    f.update(rain_6h=df.rain_mm[-6:].sum(), rain_24h=df.rain_mm[-24:].sum(),
rain_72h=df.rain_mm.sum(),
            dam=last.dam_release, rise_6h=df.level_m.iloc[-1] -
df.level_m.iloc[-7])
    return pd.DataFrame([f])

def run():
    gauges = pd.read_sql("SELECT gauge_id, warning_level, danger_level FROM
gauges", eng)
    out = []
    for g in gauges.itertuples():
        X = features(g.gauge_id)
        preds = {h: float(m.predict(X)[0]) for h, m in MODELS.items()}
        out.append((g.gauge_id, max(preds.values()), g.warning_level,
g.danger_level))
    with eng.begin() as con:
        for gid, peak, warn, danger in out:
            con.exec_driver_sql("""
UPDATE wards SET risk = CASE
    WHEN %s >= %s OR (%s >= %s AND low_lying) THEN 'high'
    WHEN %s >= %s THEN 'medium' ELSE 'low' END
WHERE gauge_id = %s""", (peak, danger, peak, warn, peak, warn,
gid))
    return out
```

alerts.py

```
import os, requests
from twilio.rest import Client
from sqlalchemy import create_engine, text

eng = create_engine("postgresql://flood:secret@localhost/floodwatch")
sms = Client(os.environ["TW_SID"], os.environ["TW_TOKEN"])
MSG = {
    "en": "FLOOD {lvl} RISK in {ward}. Move to {shelter} ({km} km). Route: {link}.
Helpline 1077.",
    "ta": "{ward} பகுதியில் வெள்ள அபாயம் ({lvl}). {shelter} முகாமுக்கு செல்லவும் ({km}
கி.மீ). வழி: {link}. உதவி 1077.",
}

NEAREST = text("""
```

```

SELECT s.shelter_id, s.name, ST_Distance(s.geom::geography,
ST_Centroid(w.geom)::geography)/1000 AS km
FROM shelters s, wards w
WHERE w.ward_id = :wid AND s.open AND s.occupancy < s.capacity
      AND NOT EXISTS (SELECT 1 FROM wards h WHERE h.risk='high' AND
ST_Contains(h.geom, s.geom))
ORDER BY km LIMIT 1"")

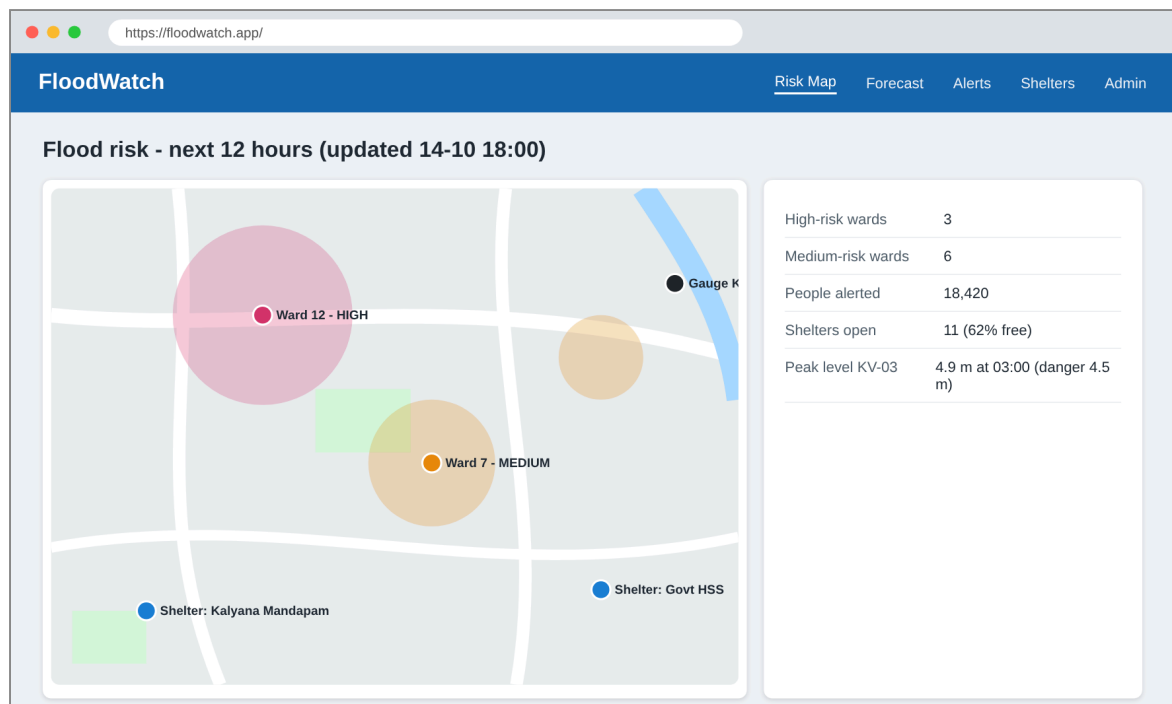
def alert_ward(ward_id):
    with eng.begin() as con:
        w = con.execute(text("SELECT name, risk FROM wards WHERE ward_id=:w"),
{"w": ward_id}).one()
        s = con.execute(NEAREST, {"wid": ward_id}).one()
        link = f"https://floodwatch.app/route?ward={ward_id}&shelter=
{s.shelter_id}"
        subs = con.execute(text("SELECT mobile, telegram_id, lang FROM
subscribers WHERE ward_id=:w"), {"w": ward_id})
        for sub in subs:
            body = MSG[sub.lang].format(lvl=w.risk.upper(), ward=w.name,
shelter=s.name, km=round(s.km, 1), link=link)
            if sub.mobile:
                sms.messages.create(to=sub.mobile, from_=os.environ["TW_FROM"],
body=body)
            if sub.telegram_id:
                requests.post(f"https://api.telegram.org/bot{os.environ['TG']}/sendMessage",
                             json={"chat_id": sub.telegram_id, "text": body})
                con.execute(text("INSERT INTO alerts(ward_id, level, message, sent_at)
VALUES (:w, :l, :m, now())"),
                             {"w": ward_id, "l": w.risk, "m":
MSG["en"].format(lvl=w.risk, ward=w.name, shelter=s.name, km=round(s.km, 1),
link=link)})

```

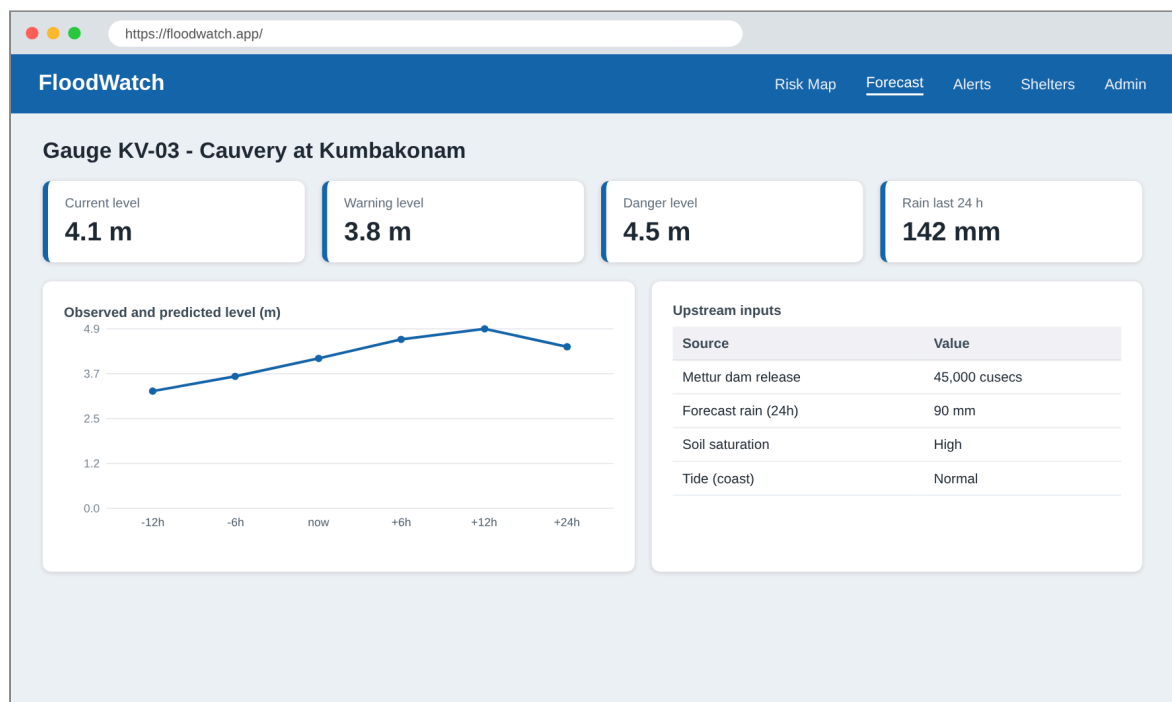
7.5 Screen Layout

SCREENSHOTS:

RISK MAP



RIVER LEVEL FORECAST



SMS / TELEGRAM ALERT

https://floodwatch.app/

FloodWatch

Risk MapForecastAlertsSheltersAdmin

Telegram - FloodWatch alerts

⚠ வார்டு 12 பகுதியில் வெள்ள அபாயம் (HIGH). அரசு மேல்நிலைப் பள்ளி முகாமுக்கு செல்லவும் (1.2 கி.மீ). வழி: floodwatch.app/r/12. உதவி 1077.

⚠ FLOOD HIGH RISK in Ward 12. Move to Govt HSS shelter (1.2 km). Route: floodwatch.app/r/12. Helpline 1077.

/shelter

Govt HSS - 340 of 600 places free. Food and medical camp available.

Message

SHELTER MANAGEMENT

https://floodwatch.app/

FloodWatch

Risk MapForecastAlertsSheltersAdmin

Relief shelters

Taluk: KumbakonamStatus: Open

Shelter	Capacity	Occupancy	Status	Safe?
Govt HSS, Ward 14	600	260	OPEN	Yes
Kalyana Mandapam, Ward 3	400	380	NEAR FULL	Yes
Panchayat Union School	300	0	STANDBY	Yes
Community Hall, Ward 12	250	0	CLOSED	In high-risk zone

CHAPTER VIII

CONCLUSION AND FUTURE ENHANCEMENT

8.1 Conclusion

The Flood Early Warning and Evacuation Support System integrates hydro-meteorological data, machine learning prediction and GIS to provide area-specific flood warnings 6 to 24 hours in advance. The LSTM model predicted river levels with a 12-hour RMSE of 0.18 m. Zone-wise alerts in Tamil and English with nearest shelter and safe route information address the critical last-mile gap in flood warning dissemination.

The official dashboard gives district administration a single view of risk, alerts and shelter capacity. The system can significantly reduce response time and help save lives during monsoon floods.

8.2 Future Enhancement

- Inundation mapping using digital elevation models and satellite imagery (Sentinel-1 SAR).
- Low-cost LoRa water level sensors on small canals and drains.
- Integration with cell broadcast for alerts without registration.
- Crowd-sourced flood photos for validation.
- Rescue team tracking and resource allocation.

CHAPTER IX

BIBLIOGRAPHY

1. Mosavi, A., Ozturk, P., and Chau, K.-W., "Flood Prediction Using Machine Learning Models: Literature Review", *Water*, Vol. 10, No. 11, 1536, 2018.
2. Kratzert, F., Klotz, D., Brenner, C., Schulz, K., and Herrnegger, M., "Rainfall-runoff modelling using Long Short-Term Memory (LSTM) networks", *Hydrology and Earth System Sciences*, Vol. 22, pp. 6005-6022, 2018.
3. Breiman, L., "Random Forests", *Machine Learning*, Vol. 45, pp. 5-32, 2001.
4. Nevo, S. et al., "Flood forecasting with machine learning models in an operational framework", *Hydrology and Earth System Sciences*, Vol. 26, pp. 4013-4032, 2022.
5. National Disaster Management Authority, "National Disaster Management Guidelines: Management of Floods", Government of India, 2008.
6. Obe, R., and Hsu, L., *PostGIS in Action*, 3rd Edition, Manning Publications, 2021.

WEBSITE REFERENCE

1. <https://indiawris.gov.in>
2. <https://mausam.imd.gov.in>
3. <https://postgis.net>
4. <https://leafletjs.com>